

D3.1 System Architecture

Due date: **31/01/2024**
Submission Date: **24/01/2024**
Revision Date: **26/06/2026**

Start date of project: **01/07/2023**

Duration: **36 months**

Lead organisation for this deliverable: **Carnegie Mellon University Africa**

Responsible Person: **D. Vernon**

Revision: **2.7**

Project funded by the African Engineering and Technology Network (Afretec) Inclusive Digital Transformation Research Grant Programme		
Dissemination Level		
PU	Public	PU
PP	Restricted to other programme participants (including Afretec Administration)	
RE	Restricted to a group specified by the consortium (including Afretec Administration)	
CO	Confidential, only for members of the consortium (including Afretec Administration)	

Executive Summary

This deliverable represents the outcome of Task 3.1. It specifies the CSSR4Africa system architecture in detail, identifying the component subsystems, the modules (i.e., ROS nodes) comprising each subsystem, and the information exchanged between subsystems and modules. This includes a specification of the data that are input to each module, the data that are output from each module, and the data that are used to control the operation of the module, including the manner in which this data is made available or accessed: through ROS topics, services, or actions, or through input and configuration files.

Based on the ROS nodes identified in Deliverables D2.2 and D2.3, Deliverable D3.1 provides the requirements for work package WP4 and WP5 on robot sensing and action, complementing and augmenting the detailed specification already provided in the work plan, as well as providing the architecture for the integration of the ROS nodes in each subsystem into a complete operational robot control system.

Contents

1	Introduction	4
2	System Architecture Overview	5
3	ROS Node Specifications	6
3.1	Animate Behavior	6
3.2	Robot Mission Interpreter	9
3.3	Face Detection	12
3.4	Gesture Execution	15
3.5	Overt Attention	19
3.6	Person Detection	23
3.7	Robot Localization	26
3.8	Robot Navigation	28
3.9	Sound Detection	31
3.10	Speech Event	33
3.11	Tablet Event	36
3.12	Text to Speech	38
4	System Architecture in Detail	40
	References	41
	Principal Contributors	42
	Document History	43

1 Introduction

This deliverable, D3.1 System Architecture Design, provides a detailed specification of the architecture of the CSSR4Africa software system that controls the culturally sensitive social interaction between the Pepper robot and a human, i.e., a visitor in one of the two use case scenarios specified in Deliverable D2.1.

It specifies the architecture's component subsystems, the modules (i.e., ROS nodes) comprising each subsystem, and the information exchanged between each module. This includes a specification of the data that are input to each module, the data that are output from each module, and the data that are used to control the operation of the module, including the manner in which this data is made available or accessed: through ROS topics, services, or actions, or through input and configuration files.

Based on the ROS nodes identified in Deliverables D2.2 and D2.3, Deliverable D3.1 provides the requirements for work package WP4 and WP5 on robot sensing and action, complementing and augmenting the detailed specification already provided in the work plan, as well as providing the architecture for the integration of the ROS nodes in each subsystem into a complete operational robot control system.

For each ROS node, the deliverable specifies the node configuration parameters that are read from the associated configuration file, the data that are read from the associated input file, the data that are written to an associated output file, the ROS topics to which the node subscribes for input, the ROS topics to which the node publishes output, the services that will be advertised and served, and the services that will be invoked.

The dynamics of the interaction between the visitor and the robot, implemented with these ROS nodes, will be specified in Deliverable D5.4.2 Robot Mission Language, and implemented in the `behaviorController` node in Deliverable D5.4.3 Robot Mission Interpreter.

In the following, we provide a high-level overview of the system architecture, a specification of each constituent ROS node, and detailed architecture that includes the topics published by each node, the topics each node subscribes to, and the services advertised and served by each node.

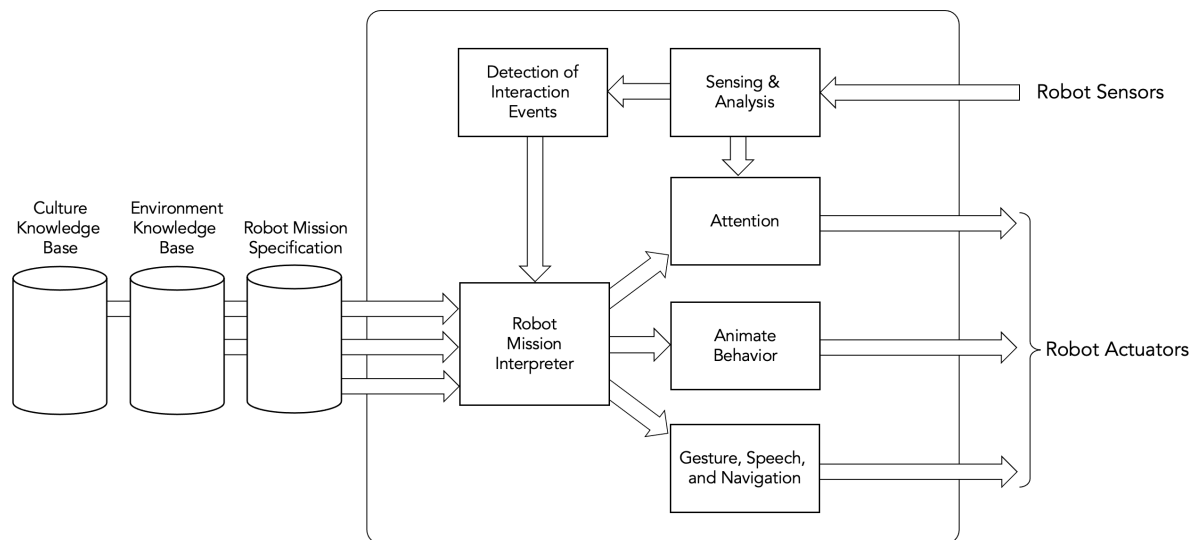


Figure 1: The CSSR4Africa system architecture.

2 System Architecture Overview

The CSSR4Africa system architecture comprises six subsystems, as follows; also see Figure 1.

1. Sensing and Analysis
2. Detection of Interaction Events
3. Animate Behavior
4. Attention
5. Robot Mission Interpreter
6. Gesture, Speech, & Navigation

Each subsystem comprises one or more ROS nodes, as shown in Table 1, with thirteen ROS nodes in total. We provide a detailed specification of each ROS node in the following section. All thirteen nodes are part of the `cssr_system` ROS package; see Deliverable D3.2 Fig. 4 and Table 1.

Subsystem	ROS Node
Sensing and Analysis	faceDetection personDetection soundDetection
Detection of Interaction Events	speechEvent tabletEvent
Robot Mission Interpreter	behaviorController
Attention	overtAttention
Animate Behavior	animateBehavior
Gesture, Speech, & Navigation	gestureExecution robotLocalization robotNavigation textToSpeech

Table 1: The thirteen ROS nodes that comprise the six subsystems in the CSSR4Africa culturally-sensitive human-robot interaction software system.

3 ROS Node Specifications

3.1 Animate Behavior

ROS Node Name

`animateBehavior`

Functional Specification

This ROS node gives the robot the appearance of an animate agent by continually making subtle body movements, flexing its hands a little, and rotating its base slightly.

The node actuates the robot joints periodically in a random pattern, keeping the joint angles close to the default home positions. All the joints, except `headYaw` and `headPitch`, are actuated, as well as the wheels to effect rotation about the robot's z -axis, but not forward movement along the robot's x -axis. The `headYaw` and `headPitch` joints are actuated by the `attention` node. The extent of the movements is determined by an external parameter. Specifically, the range of movement, from which the actual movement will be a random sample, will be specified as a percentage of one tenth the full range of movement. Thus, 10% means that the amount of motion will vary randomly between the home value and half the maximum value, assuming the home value is midway between the minimum and maximum values.

The names of the topics to be used for each actuator are read from a data file comprising a sequence of key-value pairs. The key is the name of the actuator. The value is the topic name. There are two data files, one for the physical robot and another for the simulator.

To ensure that the robot does not make these animate movements when engaged in culturally sensitive social interaction through gestures, speech, or when navigating, the generation of animate behavior is enabled and disabled by the Robot Mission Interpreter subsystem using a dedicated ROS service which the `animateBehavior` node advertizes and serves.

Any of the three types of animate behavior — body movement, hand flex, and rotation — can be selectively invoked. All three will be invoked if none are selectively invoked.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `animateBehavior` node is determined by the contents of a configuration file, `animateBehaviorConfiguration.ini`, that contains a list of key-value pairs as shown below.

Key	Values	Effect
platform	robot, simulator	Specifies the platform on which the node is to be run, i.e., the physical Pepper robot or the Pepper simulator
behavior	body, hands, rotation	Specifies the type of animate behavior to exhibit.
range	<number>	Specifies the range of actuator movement as a percentage of half the full range of movement.
robotTopics	pepperTopics.dat	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
simulatorTopics	simulatorTopics.dat	Specifies the filename of the file in which the simulator sensor and actuator topic names are stored.
verboseMode	true, false	Specifies whether diagnostic data is to be printed to the terminal.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node does not subscribe to any topics.

Topics Published

This node publishes on several actuator topics which are identified in the file given by the `robotTopics` and `simulatorTopics` key-value pairs in the configuration file. The following are the topics to which the `animateBehavior` node publishes.

Topic	Actuator	Platform
/pepper_dcm/RightHand_controller/ follow_joint_trajectory	RHand	Physical robot
/pepper_dcm/LeftHand_controller/ follow_joint_trajectory	LHand	Physical robot
/pepper_dcm/RightArm_controller/ follow_joint_trajectory	RShoulderPitch, RShoulderRoll, RElbowYaw, RElbowRoll, RWristYaw	Physical robot
/pepper_dcm/LeftArm_controller/ follow_joint_trajectory	LShoulderPitch, LShoulderRoll, LElbowYaw, LElbowRoll, LWristYaw	Physical robot
/pepper_dcm/Pelvis_controller/ follow_joint_trajectory	HipRoll, HipPitch, KneePitch	Physical robot
/cmd_vel	WheelFL, WheelFR, WheelB	Physical robot
/pepper/RightArm_controller/ follow_joint_trajectory	RElbowYaw, RElbowRoll	Simulator
/pepper/LeftArm_controller/ follow_joint_trajectory	LElbowYaw, LElbowRoll	Simulator
/pepper/Pelvis_controller/ follow_joint_trajectory	HipRoll, HipPitch, KneePitch	Simulator
/pepper/cmd_vel	WheelFL, WheelFR, WheelB	Simulator

Services Supported

This node provides and advertizes a server for a service `/animateBehavior/set_activation` to enable and disable the operation of the node, i.e., to activate or suspend the publishing of data on the actuator topics to give the appearance of an animate agent. It uses a package-specific msg, `State.msg` with just one field string `state`, with a value of either “enabled” or “disabled”. Depending on the string value, `animateBehavior` will be enabled or disabled. If the enable/disable request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `behaviorController` node, enabling or disabling animate behavior, as needed.

The following summarizes the services supported.

Service	Message Value	Effect
<code>/animateBehavior/set_activation</code>	enabled, disabled	Enable or disable animate behavior

Services Called

This node does not call any services.

3.2 Robot Mission Interpreter

ROS Node Name

`behaviorController`

Functional Specification

This ROS node provides that interprets the robot mission language developed in Task 5.4.2 and implements the specification of the two use case scenarios in that language, as documented in Deliverable D5.4.2. It does so by recruiting the robot and visitor behaviors documented in Deliverables D2.2 and D2.3, and the cultural knowledge encapsulated in Deliverable D5.4.1, and realized in the robot sensing modules developed in Work Package 4 and the robot behavior modules developed in Work Package 5. The outcome of this task is described in Deliverable D5.4.3.

The dynamics of interaction between the robot and the visitor in the two use case scenarios are specified by robot mission specifications written in the robot mission language defined in Task 5.4.2. The `behaviorController` node interacts with other CSSR4Africa ROS nodes to realize these dynamics. It does this by calling various services, subscribing to topics and sending action goals. These are detailed below.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `behaviorController` node is determined by the contents of a configuration file, `behaviorControllerConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
<code>scenarioSpecification</code>	<code><specificationFile></code>	Specifies the name of the robot mission specification file. The node will look for an xml file, excluding extension, with that name from the data folder.
<code>verboseMode</code>	<code>true, false</code>	Specifies whether diagnostic data is to be printed to the terminal.
<code>asrEnabled</code>	<code>true, false</code>	Specifies whether Automatic Speech Recognition is enabled on the platform or not.
<code>audioMode</code>	<code>true, false</code>	Specifies whether vocalization of audio debugging messages should be enabled or not

Input Data File

This node reads the interaction scenario robot mission specification file specified in the configuration file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to these topics, as follows.

Topic	Node	Platform
/overtAttention/data	/overtAttention	Physical robot & Simulator

Topics Published

This node does not publish to any topics.

Services Supported

This node does not provide and advertize any services.

Services Called

This node calls the following services to implement the interaction dynamics specified in the scenario robot mission specification.

Service	Message Format	Effect
animateBehaviour/set_activation	state	Enable or Disable the Animate Behavior mechanism.
gestureExecution/perform_gesture	gesture_type, gesture_id, arm, deictic_shape, gesture_duration, bow_nod_angle, location_x, location_y, location_z	Invokes the gesture subsystem to perform a type of gesture with specified coordinates. The coordinates are only necessary if the gesture_type is deictic.
overtAttention/set_mode	state, location_x, location_y, location_z	Sets the mode of attention for the Overt Attention node to follow.
robotLocalization/set_pose	x, y, theta	Informs the localization subsystem of the current pose of the robot
robotNavigation/set_goal	goal_x, goal_y, goal_theta	Invokes the navigation subsystem to move to the specified coordinates.
speechEvent/set_language	language	Sets the language for the ASR to transcribe utterances to
tabletEvent/prompt_and_get_response	message	Sends the text to be printed as a message on the Tablet.
textToSpeech/say_text	message, language	Sends the text to be converted to an audio signal based on the language specified and played on the robot's loudspeakers.

Action goals sent

The node interacts with the action servers detailed in the table below. Each entry specifies the action server name, the message format required, and the expected effects or observations.

Service	Message Format	Effect
gestureExecution/perform_gesture	gesture_type, gesture_id, arm, deictic_shape, gesture_duration, bow_nod_angle, location_x, location_y, location_z	Invokes the gesture sub-system to perform a type of gesture with specified coordinates. The coordinates are only necessary if the gesture_type is deictic.
robotNavigation/set_goal	goal_x, goal_y, goal_theta	Invokes the navigation sub-system to move to the specified coordinates.
speechEvent/recognise_speech_action	language	Sets the language for the ASR to transcribe utterances to
textToSpeech/say_text	message, language	Sends the text to be converted to an audio signal based on the language specified and played on the robot's loudspeakers.

Helper Classes Used

This node uses an instantiation of a helper class `CultureKnowledgeBase` to read the culture knowledge base file and retrieve the required data using class access methods.

Similarly, it uses an instantiation of a helper class `EnvironmentKnowledgeBase` to read the environment knowledge base file and retrieve the required data using class access methods.

The type of the variables that are passed as arguments to the `CultureKnowledgeBase` helper class access methods and the `EnvironmentKnowledgeBase` helper class access methods have not yet been defined. These will be done when the helper classes are fully specified in Deliverables D5.4.1 and D5.4.2, respectively.

3.3 Face Detection

ROS Node Name

faceDetection

Functional Specification

This ROS node detects faces and their location in the field of view of the Pepper robot and determine their gaze direction. It computes their position in an 2D image frame of reference. In addition, the region that the face occupies in the image is determined by computing the bounding box surrounding the face. If more than one face is present in the robot's field of view, then all of them are detected and localized.

To ensure coherence in detection and localization over time, each detected face is labelled (e.g., "Face 1") and the same label is assigned to that person in subsequent images. The label and the bounding box are colour-coded, assigning different colours to different faces, and the same colour to a given face in each image in a sequence of images. If that face is no longer detected in one or more images (the number to be specified in a configuration parameter value) due to, for example, a false reject error, then that label is not reused. If that face reappears in a subsequent image, it is given a new label. As such, this module is only concerned with consistent detection of face over time, not recognition of previously detected face, and it is assumed that people don't change between images. Consequently, a face detected in one image is deemed to be the same as one in a previous image if the spatial displacement between their centroids is within a defined tolerance or the Intersection over Union (IoU) between their bounding boxes exceeds a specified threshold defined in the configuration parameters.

Additionally, for each detected face, the ROS node assesses whether the person is gazing directly at the robot. This is accomplished using head pose estimation, which provides a simpler alternative to mutual gaze detection.

The node has two inputs: an RGB image and a depth image from either the robot's cameras or the Intel realsense camera. In addition, there is an option to use compressed images instead of raw image for both RGB and depth image.

The node has three outputs: an RGB image, with bounding boxes drawn around each detected face in the RGB images, depth image and an array of records (one record for each face detected).

Each record consists of the face label, the 3D image coordinates representing the centroid of the bounding box, the distance (or depth) of the face from the camera based on the centroid, a Boolean value indicating whether mutual gaze is established between the corresponding face and the Pepper robot, and the width and height of the face bounding box.

The image is displayed in an openCV window when the node is operating in verbose mode. The array of records is published to a topic named `/faceDetection/data`.

The names of the topics to be used for each camera is read from a data file comprising a sequence of key-value pairs. The key is the name of the camera. The value is the topic name.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal, and output images are also displayed in openCV windows.

Configuration File

The operation of the `faceDetection` node is determined by the contents of a configuration file, `faceDetectionConfiguration.json`, that contain a list of key-value pairs as shown below.

<code>algorithm</code>	<code>mediapipe, sixdrep</code>	Specifies which algorithm to use.
<code>useCompressed</code>	<code><true or false></code>	Specifies to use compressed images or raw images.
<code>mpFacedetConfidence</code>	<code><number></code>	Specifies the confidence threshold for the MediaPipe face detection algorithm.
<code>mpHeadposeAngle</code>	<code><number></code>	Specifies the maximum angular deviation (in degrees) for MediaPipe head pose estimation.
<code>centroidMaxDistance</code>	<code><number></code>	Specifies the maximum allowed distance (in pixels) between centroids for tracking continuity.
<code>centroidMaxDisappeared</code>	<code><number></code>	Specifies the maximum number of frames a centroid can disappear before being considered lost.
<code>sixdrepnetConfidence</code>	<code><number></code>	Specifies the confidence threshold for the SixDRepNet pose estimation algorithm.
<code>sixdrepnetHeadposeAngle</code>	<code><number></code>	Specifies the maximum angular deviation (in degrees) for SixDRepNet head pose estimation.
<code>sortMaxDisappeared</code>	<code><number></code>	Specifies the maximum number of frames an object can disappear for SORT tracker before being removed.
<code>sortMinHits</code>	<code><number></code>	Specifies the minimum number of consecutive hits required for SORT tracker initialization.
<code>sortIouThreshold</code>	<code><number></code>	Specifies the Intersection over Union (IoU) threshold for SORT tracker associations.
<code>verboseMode</code>	<code><true or false></code>	Specifies whether diagnostic data is to be printed to the terminal and diagnostic images are to be displayed in OpenCV windows.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to four topics: two RGB camera sensor topics (one published by the Pepper robot, one by the RealSense camera), and two depth camera topics (one published by the Pepper robot, one by the RealSense camera). These are specified in the files identified by the `robotTopics` key-value pair in the configuration file.

The following are the topics to which the `faceDetection` node subscribes.

Topic	Sensor / Node	Platform
<code>/camera/color/image_raw</code>	RGBRealSense	Intel RealSense
<code>/camera/color/image_raw/compressed</code>	RGBRealSense (Compressed)	Intel RealSense
<code>/camera/aligned_depth_to_color/image_raw</code>	DepthRealSense	Intel RealSense
<code>/camera/aligned_depth_to_color/image_raw/compressed</code>	DepthRealSense (Compressed)	Intel RealSense
<code>/naoqi_driver/camera/front/image_raw</code>	FrontCamera	Physical Robot (Pepper)
<code>/naoqi_driver/camera/depth/image_raw</code>	DepthCamera	Physical Robot (Pepper)

Topics Published

The following are the topics which the `faceDetection` node publishes.

Topic	Sensor / Actuator / Node	Platform
<code>/faceDetection/data</code>	<code>overtAttention</code> , GUI	Physical robot

Services Supported

This node does not support any services.

Services Called

This node does not call any services.

3.4 Gesture Execution

ROS Node Name

`gestureExecution`

Functional Specification

This ROS node provides the robot with the ability to execute five forms of gesture: deictic, symbolic, and iconic non-verbal hand gestures, and bowing and nodding body gestures.

The specifications for these gestures are in joint space, except for deictic gestures which are in Cartesian space. Some gestures, e.g., iconic and symbolic hand gestures, are specified by learning the required motions either by manual teleoperation, recording the joint angles, or by demonstration, using an RGB-D depth camera to determine the joint angles of human gestures in a skeletal model and mapping these to the robot joints. Other gestures, i.e., deictic hand gestures and body gestures, are specified by gesture parameters, such as the pointing location for deictic gestures and the degree of inclination for bowing and nodding, and the joint angles are computed using the kinematic model of the robot head, torso, and arms. For deictic gestures, which require the robot to point at objects in its environment and move its head to look at the object, the pose of the robot in the world frame of reference is also used.

If the arm cannot achieve the required pose for a deictic gesture, the robot rotates to make the pose achievable, returning to the original orientation once the gesture is complete. Furthermore, the arm returns to a neutral position by the robot's side when the gesture is complete.

Iconic and symbolic gestures are defined by descriptors that specify the final gesture joint configuration and the manner in which that configuration is achieved. Descriptors comprise four elements. Each element is a key-value pair, where the value can be an identifier, a number, a vector of numbers, or a vector of a vector of numbers.

The first key-value pair specifies the gesture type (e.g., `type iconic`, `type symbolic`).

The second key-value pair identifies the ID number (e.g., `ID 01`).

The third element defines the number of waypoints in the trajectory, including the start gesture joint configuration and the final gesture joint configuration.

The fourth element is a vector of joint angles vectors. The number of joint angle vectors is equal to the number of way points, including the start joint configuration and the final gesture configuration. Body gestures have three joints: knee pitch, hip pitch, hip roll. Iconic and symbolic gestures have five joints: shoulder pitch, shoulder roll, elbow yaw, elbow roll, and wrist yaw. Before beginning the gesture, the arm is moved from its current joint configuration to the start joint configuration, i.e., the joint angles specified in the first vector in the vector of vector of joint angles.

The number of elements in the vector of joint angles is determined by the gesture type.

Descriptors for each gesture are stored in an external descriptor file.

If an iconic or symbolic gesture involves two arms, they are treated as a composite of two individual gestures, one for each arm.

The joint angles for bow and nod body gestures, as well as hand deictic gestures, are computed at run time using the kinematic model of the robot and the bow angle, nod angle, or the location in the environment to which the robot should point. The bow angle, nod angle, and pointing location are provided as an input to the module, along with the time in milliseconds that should elapse between the start of the gesture and the end of the gesture.

The pointing location with respect to the robot body, specified by the shoulder pitch and shoulder roll angles, is computed from the pointing location in the world frame of reference (and supplied as an input to the module) and the pose of the robot in the world frame of reference (provided by the `robotLocalization` node). No waypoints are required for deictic gestures; the joints are actuated to achieve the target joint angles, interpolating linearly, or adjusting the joint angles, joint angular velocities, and joint accelerations to mimic biological movement by using a minimum jerk model of biological motion.

It is assumed that the knee pitch angle is fixed during a bow body gesture and that the bow angle corresponds to the change in the hip pitch angle with respect to the default hip pitch angle. Similarly, it is assumed that the

nod angle is the change in the head pitch angle with respect to the default head pitch angle. Finally, it is assumed that the arm and fingers are straight in a deictic gesture, with fixed values of elbow yaw, elbow roll, wrist yaw, and hand angles, so that the palm of the hand is directed upwards.

The input to the module is a record comprising the gesture type (e.g., *iconic*, *symbolic*, *deictic*, *bow*, *nod*), the gesture ID for symbolic or iconic gestures (e.g., 01), the duration of the gesture in milliseconds, and either a bow angle in degrees (for a bow body gesture), or a nod angle in degrees (for a nod body gesture), or the three dimensional coordinates of a pointing location (for a deictic gesture). For deictic gestures, the module also inputs the current robot pose from the `robotLocalization` node.

The output is a sequence of joint angles, joint angular velocities, and, optionally, joint angular accelerations. Data is published on the appropriate topics, as required.

The names of the topics to be used for each actuator is read from a data file comprising a sequence of key-value pairs. The key is the name of the actuator. The value is the topic name.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `gestureExecution` node is determined by the contents of a configuration file, `gestureExecutionConfiguration.ini`, that contains a list of key-value pairs, as shown below.

Key	Values	Effect
<code>interpolation</code>	<code>linear</code> , <code>biological</code>	Specifies the interpolation type. This indicates how the joint angles that define the trajectory in joint space between the current joint angles and the gesture joint angles are computed for body gesture and hand deictic gestures, and between way points for iconic and symbolic gestures. The two options are: (a) independent linear interpolation of each joint angle, and (b) biological motion, selecting the sequence of joint angular velocities and joint accelerations to form a trajectory in time and joint space that mimics biological movement.
<code>gestureDescriptors</code>	<code>gestureDescriptors.dat</code>	Specifies the filename of the file in which the gesture descriptors are stored.
<code>robotTopics</code>	<code>pepperTopics.dat</code>	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
<code>verboseMode</code>	<code>true</code> , <code>false</code>	Specifies whether diagnostic data is to be printed to the terminal.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to one topic, published by `robotLocalization` node, which provides the pose of the robot.

The following are the topics to which the `gestureExecution` node subscribes.

Topic	Node	Platform
<code>/robotLocalization/pose</code>	<code>robotLocalization</code>	Physical robot

Topics Published

The following are the topics to which the `gestureExecution` node publishes. These are specified in the file identified by the `robotTopics` key-value pair in the configuration file.

Topic	Actuator	Platform
<code>/pepper_dcm/RightHand_controller/follow_joint_trajectory</code>	RHand	Physical robot
<code>/pepper_dcm/LeftHand_controller/follow_joint_trajectory</code>	LHand	Physical robot
<code>/pepper_dcm/RightArm_controller/follow_joint_trajectory</code>	RShoulderPitch, RShoulderRoll, RElbowYaw, RElbowRoll, RWristYaw	Physical robot
<code>/pepper_dcm/LeftArm_controller/follow_joint_trajectory</code>	LShoulderPitch, LShoulderRoll, LElbowYaw, LElbowRoll, LWristYaw	Physical robot
<code>/pepper_dcm/Pelvis_controller/follow_joint_trajectory</code>	HipRoll, HipPitch, KneePitch	Physical robot
<code>/cmd_vel</code>	WheelFL, WheelFR, WheelB	Physical robot

Services Supported

This node provides and advertizes a server for a service `/gestureExecution/perform_gesture` to initiate the performance of a required gesture. The message has several fields, as follows.

Field	Field Value	Field Type	Units
<code>gesture_type</code>	iconic, symbolic deictic, bow, nod	String	
<code>gesture_id</code>	<number>	Integer	
<code>arm</code>	LEFT, RIGHT, EITHER	String	
<code>deictic_shape</code>	PALM.UPWARDS, PALM.DOWNWARDS	String	
<code>gesture_duration</code>	<number>	Integer	milliseconds
<code>bow_nod_angle</code>	<number>	Integer	degrees
<code>location_x</code>	<number>	Real	metres
<code>location_y</code>	<number>	Real	metres
<code>location_z</code>	<number>	Real	metres

If the `perform_gesture` request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `behaviorController` node.

Services Called

This node invokes the service `/overtAttention/set_mode` to move the robot’s head to focus on the location required to gesture at.

Service	Message Value	Effect
<code>/overtAttention/set_mode</code>	location x, y, z	Move the head to focus on a location defined by the parameters

Actions Supported

This node provides and advertises an action server at `/gestureExecution/perform.gesture`, which is used to initiate the execution of a specified gesture. It relies on a package-specific action definition, `gesture.action`. The action definition includes the following fields:

Field	Field Value	Field Type	Units
<code>gesture_type</code>	<code>iconic</code> , <code>symbolic</code> <code>deictic</code> , <code>bow</code> , <code>nod</code>	String	
<code>gesture_id</code>	<number>	Integer	
<code>arm</code>	<code>LEFT</code> , <code>RIGHT</code> , <code>EITHER</code>	String	
<code>deictic_shape</code>	<code>PALM.UPWARDS</code> , <code>PALM.DOWNWARDS</code>	String	
<code>gesture_duration</code>	<number>	Integer	milliseconds
<code>bow_nod_angle</code>	<number>	Integer	degrees
<code>location_x</code>	<number>	Real	metres
<code>location_y</code>	<number>	Real	metres
<code>location_z</code>	<number>	Real	metres

3.5 Overt Attention

ROS Node Name

`overtAttention`

Functional Specification

This ROS node provides the robot with the ability to direct the gaze of the robot to salient features in the environment or to a given location in the environment.

There are five modes of operation: a social mode, a scanning mode, a location mode, a seeking mode, and a disabled mode. The first is active when the robot is engaged in social interaction. In this mode, the salient features are people's bodies, faces, eyes, and voices. The second is active when the robot is not engaged in social interaction and it is scanning the environment looking for people with whom to interact. In this mode, the salient features also include objects of interest in the robot's environment, in addition faces. People's faces has a higher priority than other general features. In the second mode of operation, the robot switches its focus of attention after a short period of time and does not return directly to the original focus of attention, thereby scanning its environment. In the third mode of operation, the robot gazes at a given location in its environment. If the head cannot achieve the required pose to gaze at a given location, the robot rotates to make the pose achievable. In the fourth mode, seeking, the robot attempts to establish mutual gaze with a person nearby for a short duration, after which, if unsuccessful, it returns success or failure. To establish mutual gaze, the node searches for a face that is looking straight at the robot. The robot might have to rotate about its base as well as rotating its head when searching. In disabled mode, the head is centred and remains immobile. The mode of operation is selected on the basis of a service request (see below).

Two saliency maps are generated, one based on social features and one based on the combination of social and general conspicuous features. The first is based on the output of Task 4.2.2 Face & Mutual Gaze Detection and Localization, and Task 4.2.3 Sound Detection and Localization. The second is based on information-theoretic saliency and the output of Task 4.2.2 Face & Mutual Gaze Detection and Localization [1].

In the scanning attention mode, three processes are active. The first is a winner-take-all process to determine a single focus of attention from the candidates in the saliency map. This is effected by a selective tuning model [2, 3, 4]. The second is an Inhibition-of-Return (IOR) mechanism that attenuates the attention value of previous winning locations so that new regions become the focus of attention. The third is an habituation process to gradually reduce the salience of the current focus of attention, thereby ensuring that attention is fixated on a given point only for a limited period [5].

The robot's gaze is directed by publishing the appropriate messages on the topic that controls the `headYaw` and `headPitch` joints so that the gaze is centred on the focus of attention. This requires the calibration of the x and y offset of the focus of attention in the image to the change in `headYaw` and `headPitch` angles, respectively. In the case of aural attention to conspicuous sounds, it requires the calibration of the angle of arrival of the sound with the change in `headYaw` angle. Fixation on sounds will only control the `headYaw` angle, i.e., rotation in the horizontal plane about the head's z -axis.

These calibration constants are provided as parameters to the node. If the angle of rotation of the `headYaw` joint is greater than some threshold (defined as a parameter), then after rotating the head to fixate on the focus of attention, the base of the robot and the head rotate in opposite directions so that the robot continues to gaze at the focus of attention while it realigns its head with its body. The threshold for this head-torso realignment is provided as a parameter to the node.

The node has four inputs, all acquired by subscribing to the appropriate topics. These are the outputs of the `faceDetection` node and the `soundDetection` node to be used to compute the saliency map in social mode, an RGB image from the robot's top forehead camera or the additional RealSense external camera to be used to compute the saliency map in scanning mode, and, when attending to a given location in the environment, the current robot pose from the `robotLocalization` node.

The node has four outputs, effected by publishing to the topic to control the `headYaw` and `headPitch` joints, and, if required, the topic to control robot's wheels and the robot's angular velocity when adjusting its pose to recentre the gaze or enable the gaze to be directed at a given location. The third output is an RGB image depicting the saliency function and the selected focus of attention. The image is displayed in an openCV

window when the node is operating in verbose mode (see below). The fourth output is the current active mode indicating which mode of operation the node is in at that moment. The mode is continuously published to the `/overtAttention/mode` topic.

The names of the topics to be used for each actuator are read from a data file comprising a sequence of key-value pairs. The key is the name of the actuator. The value is the topic name.

To ensure that the robot does not rotate to realign its head and torso when navigating, attention is disabled by the `behaviorController` node using a dedicated service which the `overtAttention` node advertizes and serves. It can also enable attention by setting the mode to social, seeking, scanning, or location mode also effected by the `overtAttention` node using the same dedicated service which the `overtAttention` node advertizes and serves.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal, and output images are also displayed in openCV windows.

Configuration File

The operation of the `overtAttention` node is determined by the contents of a configuration file, `overtAttentionConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
<code>camera</code>	<code>FrontCamera, RealSense</code>	Specifies which RGB camera to use.
<code>realignmentThreshold</code>	<code><number></code>	Specifies the threshold on the angular difference between head and base that must be met before the head and base are realigned.
<code>xOffsetToHeadYaw</code>	<code><number></code>	Specifies the calibration constant that defines the conversion of the offset in the (horizontal) <i>x</i> -axis of an image from the image center to the change in <code>headYaw</code> joint angle.
<code>yOffsetToHeadPitch</code>	<code><number></code>	Specifies the calibration constant that defines the conversion of the offset in the (vertical) <i>y</i> -axis of an image from the image center to the change in <code>headPitch</code> joint angle.
<code>robotTopics</code>	<code>pepperTopics.dat</code>	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
<code>verboseMode</code>	<code>true, false</code>	Specifies whether diagnostic data is to be printed to the terminal and diagnostic images are to be displayed in OpenCV windows.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to six topics: three from other nodes in the `cssr_system` package, and three camera sensor topics. These are specified in the file identified by the `robotTopics` key-value pair in the configuration file.

The following are the topics to which the `overtAttention` node subscribes.

Topic	Sensor / Node	Platform
/faceDetection/data	faceDetection	Physical robot
/robotLocalization/pose	robotLocalization	Physical robot
/soundDetection/direction	soundDetection	Physical robot
/naoqi_driver/camera/front/image_raw	FrontCamera	Physical robot
/camera/color/image_raw	RGBRealSense	Intel RealSense
/camera/depth/image_rect_raw	DepthRealSense	Intel RealSense

Topics Published

This node controls five joints: `headYaw` and `headPitch` to adjust the head gaze, and `WheelFL`, `WheelFR`, and `WheelB` to rotate the robot and allow the gaze angle to be recentred. This node also publishes the current active mode of operation to a topic.

The specific topics that are used to control these joints are identified in the file given by the `robotTopics` key-value pair in the configuration file.

The following are the topics to which the `overtAttention` node publishes.

Topic	Actuator	Platform
/pepper_dcm/Head_controller/follow_joint_trajectory	headYaw, headPitch	Physical robot
/cmd_vel follow_joint_trajectory	WheelFL, WheelFR, WheelB	Physical robot
/overtAttention/mode		Physical robot

The following table details the custom message, `Status.msg`, that the attention node publishes on the `/overtAttention/mode` topic.

Field	Field Value	Field Type	Comment
mode	social, scanning, location, seek, disabled	String	
value	1 (seeking), 2 (success), 3 (failure)	Integer	

The value field only has meaning in seeking mode and scanning mode. The value 1 means the robot is attempting to establish mutual gaze. The value 2 means the robot has succeeded in detecting mutual gaze. And the value 3 means the robot failed to establish mutual gaze with anyone nearby. However, the value 3 has no meaning in scanning mode as it can keep scanning indefinitely because scanning mode is not time bound.

Services Supported

This node provides and advertizes a server for a service `/overtAttention/set_mode` to set the social, scanning, location, seeking, or disabled modes of attention. It uses a package-specific msg, `Mode.msg`. The message has four fields, as follows.

Field	Field Value	Field Type	Units
state	social, scanning, location, seeking, disabled	String	
location_x	<number>	Real	metres
location_y	<number>	Real	metres
location_z	<number>	Real	metres

If the request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `behaviorController` node and the `gestureExecution` node, setting the required mode of attention.

The following summarizes the services supported.

Service	Message Value	Effect
<code>/overtAttention/set_mode</code>	<code>social, scanning, location seeking, disabled</code>	Select mode of attention

Services Called

This node does not call any services.

3.6 Person Detection

ROS Node Name

`personDetection`

Functional Specification

This ROS node detects people in the field of view of the Pepper robot and determines their location. It computes their position in an 2D image frame of reference and in the robot's head 3D Cartesian frame of reference. In addition, the region that the person occupies in the image is determined by computing the bounding box surrounding the person. If more than one person is present in the robot's field of view, then all of them are detected and localized.

To ensure coherence in detection and localization over time, each detected person is labelled (e.g., "Person 1") and the same label is assigned to that person in subsequent images. The label and the bounding box are colour-coded, assigning different colours to different people, and the same colour to a given person in each image in a sequence of images. If that person is no longer detected in one or more images (the number to be specified in a configuration parameter value) due to, for example, a false reject error, then that label is not reused. If that person reappears in a subsequent image, she or he is given a new label. As such, this module is only concerned with consistent detection of people over time, not recognition of previously detected people, and it is assumed that people don't change between images. Consequently, a person in one image is deemed to be the same one in a previous image if the spatial displacement of the person is less than a given tolerance (to be specified by a configuration parameter value).

The node has two inputs: an RGB image from one of the robot's cameras and a depth image from one of the robot's depth sensors. In addition, there is an option to use compressed images instead of raw images for both RGB and depth image.

The node has three outputs: an RGB image and a depth image, with bounding boxes drawn around each detected person in RGB images, and an array of records, one record for each person detected.

The components of a record are the person label, the 3D coordinates corresponding to the centroid of the bounding box surrounding the person in the image, the distance from the robot to that centroid, and the width and height of the bounding box.

The RGB image and the depth image are displayed in an openCV window when the node is operating in verbose mode (see below).

The array of records is published to a topic named `/personDetection/data`.

The names of the topics to be used for each sensor will be read from a data file comprising a sequence of key-value pairs. The key is the name of the sensor. The value is the topic name.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal, and output images are also displayed in openCV windows.

Configuration File

The operation of the `personDetection` node is determined by the contents of a configuration file, `personDetectionConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
<code>useCompressed</code>	<code>true</code> or <code>false</code>	Specifies to use compressed image or raw images.
<code>confidenceIoUThreshold</code>	<code><number></code>	Specifies the confidence threshold for the YOLO person detection algorithm.
<code>sortMaxDisappeared</code>	<code><number></code>	Specifies the maximum number of frames an object can disappear for SORT tracker before being removed.
<code>sortMinHits</code>	<code><number></code>	Specifies the minimum number of consecutive hits required for SORT tracker initialization.
<code>sortIoUThreshold</code>	<code><number></code>	Specifies the Intersection over Union (IoU) threshold for SORT tracker associations.
<code>verboseMode</code>	<code>true</code> or <code>false</code>	Specifies whether diagnostic data is to be printed to the terminal and diagnostic images are to be displayed in OpenCV windows.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to five topics: four camera sensor topics and one depth sensor topic. These are specified in the file identified by the `robotTopics` key-value pair in the configuration file.

The following are the topics to which the `personDetection` node subscribes.

Topic	Sensor / Node	Platform
<code>/camera/color/image_raw</code>	RGBRealSense	Intel RealSense
<code>/camera/color/image_raw/compressed</code>	RGBRealSense (Compressed)	Intel RealSense
<code>/camera/aligned_depth_to_color/image_raw</code>	DepthRealSense	Intel RealSense
<code>/camera/aligned_depth_to_color/image_raw/compressed</code>	DepthRealSense (Compressed)	Intel RealSense
<code>/naoqi_driver/camera/front/image_raw</code>	FrontCamera	Physical Robot (Pepper)
<code>/naoqi_driver/camera/depth/image_raw</code>	DepthCamera	Physical Robot (Pepper)

Topics Published

The following are the topics to which the `personDetection` node publishes.

Topic	Sensor / Actuator / Node	Platform
<code>/personDetection/data</code>	<code>personDetection</code>	Physical robot

Services Supported

This node does not support any services.

Services Called

This node does not call any services.

3.7 Robot Localization

ROS Node Name

robotLocalization

Functional Specification

This ROS node determines the pose (position and orientation) of the robot in a Cartesian world frame of reference. It does this continuously, in real time, by updating the current pose based on relative pose estimation, using either odometry or the robot's inertial management unit IMU (or a combination of both). Since pose estimation errors using relative techniques grow with time, the module periodically resets its pose estimate using absolute pose estimation.

Absolute position estimation is accomplished by triangulation, using three landmarks (three are required because their distance from the robot is not known since the robot's range sensors are not sufficiently accurate). Landmark recognition is accomplished both using SIFT (Scale Invariant Feature Transform) and YOLO (You Look Only Once) real-time object detection. The position of the landmarks is extracted from a map of the environment. This map will be produced in Task 5.5.3 Environment Map Generation. The orientation of the robot is computed only for its rotation about the Z-axis; adjustments of body posture through rotation about the X- and Y-axes are ignored. This rotation angle is recovered by determining the direction given by the line of sight from the robot to one of the landmarks, and adjusting for any rotation about the Z-axis of the robot's head frame of reference with respect to the base frame of reference.

The node has five inputs: For relative pose estimation, the input is the odometry data published by the robot and data from the robot's accelerometer and gyrometer. For absolute pose estimation, the input takes the form of an RGB image from one of the robot's cameras. Input is also be acquired from the encoder on the head yaw actuator, i.e., the joint responsible for rotation in the azimuth (horizontal) plane. The names of the topics to be used for each sensor will be read from a data file comprising a sequence of key-value pairs. The key is the name of the sensor. The value is the topic name. The node also serves a `robotLocalization/reset_pose` service to reset the pose of the robot using absolute pose estimation. This service is typically called by the `robotNavigation` node.

The node has two outputs: an RGB image, with bounding boxes drawn around each detected landmark, and a record with the 2D pose information: x and y coordinates and rotation about the Z-axis. The image is displayed in an openCV window when the node is operating in verbose mode (see below). The record is published on a topic named `/robotLocalization/pose`.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal, and output images are also displayed in openCV windows.

Configuration File

The operation of the `robotLocalization` node is determined by the contents of a configuration file that contain a list of key-value pairs as shown below.

The configuration file is named `robotLocalizationConfiguration.ini`.

Key	Values	Effect
camera	FrontCamera, RealSense	Specifies which RGB camera to use.
resetInterval	<number>	Specifies the distance that can be travelled in centimetres before the relative pose estimate is reset using the absolute pose estimate.
robotTopics	pepperTopics.dat	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
verboseMode	true, false	Specifies whether diagnostic data is to be printed to the terminal and diagnostic images are to be displayed in OpenCV windows.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to six topics: two RGB camera sensor topics (one published by the Pepper robot, one by the RealSense camera), one depth camera topic (by the RealSense camera), an odometry topic, and inertial measurement unit (IMU) topic, and a joint states topic for the head yaw angle. These are specified in the files identified by the `robotTopics` key-value pair in the configuration file.

The following are the topics to which the `robotLocalization` node subscribes.

Topic	Sensor	Platform
/naoqi_driver/camera/front/image_raw	FrontCamera	Physical robot
/camera/color/image_raw	RGBRealSense	Intel RealSense
/camera/depth/image_rect_raw	DepthRealSense	Intel RealSense
/naoqi_driver/odom	Odometry	Physical robot
/naoqi_driver/imu/base	IMU	Physical robot
/joint_states	Head Yaw	Physical robot

Topics Published

The following are the topics to which the `robotLocalization` node publishes.

Topic	Sensor / Actuator / Node	Platform
/robotLocalization/pose	gestureExecution overtAttention robotNavigation behaviorController	Physical robot

Services Supported

This node provides and advertizes a server for a service `/robotLocalization/reset_pose` to reset the pose of the robot using absolute pose estimation. It uses a generic msg, `Reset.msg` with just one field string, with a value “reset”. If the reset request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `robotNavigation` node.

It also provides and advertizes a server for a service `/robotLocalization/set_pose` to set the pose of the robot to some given value. It uses a custom message to specify the pose with the x and y coordinates, and the angle of rotation θ about the z axis. If the set request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `behaviorController` node using data read from the environment knowledge base, typically the pose of the robot at its greeting location before it starts navigating.

The following summarizes the services supported.

Service	Message Value	Effect
/robotLocalization/reset_pose	reset	Reset robot pose
/robotLocalization/set_pose	<x> <y> <theta>	Set robot pose

Services Called

This node does not call any services.

3.8 Robot Navigation

ROS Node Name

robotNavigation

Functional Specification

This ROS node controls the locomotion of the Pepper robot so that it navigates its environment, in which there are fixed inanimate obstacles and moveable obstacles in the form of humans, from its current position along the shortest path to a destination position and orientation identified in the use case scenario robot mission specification. Navigation is effected by identifying waypoints along the navigation path and the robot moves from waypoint to waypoint.

The node optionally augments a metric workspace map of the robot's environment with obstacles corresponding to the location of any humans that have been detected in the robot's field of view by the `personDetection` node. The extent of the human obstacle is determined using culturally sensitive proxemics. This augmented workspace map is then used to generate a configuration space map that constrains the robot's path from its current location to its target location using either Dijkstra's algorithm and the A* algorithm. Waypoints are identified using one of two candidate techniques: equidistant waypoints and high path curvature waypoints. Locomotion from waypoint to waypoint is effected using one of two locomotion algorithms: Multiple Input Multiple Output (MIMO) and divide and conquer (DnQ).

The node has three inputs. The first is a record identifying the destination pose for the robot, specified by the x and y coordinates of the location and the direction the robot should face (i.e., the direction of the X-axis in the robot base frame, all specified in the workspace frame of reference). This record is part of a message either in a service call made or an action goal sent by the `behaviorController` node. The second is robot's current pose. This is provided by messages published to the `robotLocalization` node. The third input is the required cultural knowledge regarding proxemics. This is provided querying the African cultural knowledge base using an instantiation of a `CultureKnowledgeBase` helper class to read the culture knowledge base file and retrieve the required data using class access methods.

The node has two outputs. The first is sequence of forward velocity and angular velocity values published on the relevant `cmd_vel` topic. The full name of the `cmd_vel` topic is will be read from a data file comprising a sequence of key-value pairs. The second is the planned path drawn on a configuration space image. This image is displayed in an openCV window when the node is operating in verbose mode (see below).

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal, and output images are also displayed in openCV windows.

Configuration File

The operation of the `robotNavigation` node is determined by the contents of a configuration file, `robotNavigationConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
map	scenarioOneMap.dat	Specifies the filename of the file in which the workspace map is stored.
pathPlanning	Dijkstra, A*	Specifies the path planning algorithm to be used.
socialDistance	true, false	Specifies whether or not to take into consideration social constraints while navigating.
robotTopics	pepperTopics.dat	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
verboseMode	true, false	Specifies whether diagnostic data is to be printed to the terminal and diagnostic images are to be displayed in OpenCV windows.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to one topic, as follows.

Topic	Node	Platform
/robotLocalization/pose	robotLocalization	Physical robot

Topics Published

The following are the topics to which the `robotNavigation` node publishes.

Topic	Sensor / Actuator / Node	Platform
/cmd_vel	WheelFL, WheelFR, WheelB	Physical robot

Services Supported

This node provides and advertizes a server for a service `/robotNavigation/set_goal` to request navigation to a given goal position and orientation. It uses a custom message to specify the pose with the x and y coordinates, and the angle of rotation θ about the z axis. If the navigation request is successful, the service response is “1”; if it is unsuccessful, it is “0”. The service is called by the `behaviorController` node.

The following summarizes the service supported.

Service	Message Value	Effect
/robotNavigation/set_goal	<x> <y> <theta>	Define navigation goal pose

Services Called

This node calls the following two services.

Service	Message Value	Effect
/robotLocalization/reset_pose	reset	Reset the pose of the robot using absolute localization

Actions Supported

This node provides and advertises an action server at `/robotNavigation/set_goal` to request navigation to a given goal position and orientation. The goal is sent by the `behaviorController` node.

The table below summarizes the action server advertized and the message format.

Service	Message Value	Effect
/robotNavigation/set_goal	<x> <y> <theta>	Define navigation goal pose

Helper Classes Used

This node uses an instantiation of a helper class `CultureKnowledgeBase` to read the culture knowledge base file and retrieve the required data using class access methods.

Similarly, it uses an instantiation of a helper class `EnvironmentKnowledgeBase` to read the environment knowledge base file and retrieve the required data using class access methods.

The type of the variables that are passed as arguments to the `CultureKnowledgeBase` helper class access methods and the `EnvironmentKnowledgeBase` helper class access methods have not yet been defined. These will be done when the helper classes are fully specified in Deliverables D5.4.1 and D5.4.2, respectively.

3.9 Sound Detection

ROS Node Name

`soundDetection`

Functional Specification

This ROS node detects vocal sounds within the robot's hearing range and determines the direction of arrival of the sound.

Localization is limited to the azimuth (i.e., horizontal) plane. If a sound is detected, its direction of arrival will be determined in the robot's Cartesian head frame of reference based on the interaural time difference (ITD) between the arrival of the sound at the front left and front right microphones on the top of the robot's head. The node is tuned to detect human voices rather than ambient sounds or background noise by using signal processing techniques such as band-pass filtering.

The node has one inputs: the audio signal from the front left, front right, back left, and back right microphones, respectively. This data is published on `/naoqi_driver/audio`.

The node has two outputs: the angle of arrival in degrees relative to the robot head's forward-looking x -axis, and the audio signal of the detected sound captured by the front left microphone, from onset of the sound to offset.

The angle is published to a topic named `/soundDetection/direction`. The channels of the audio signal that is captured by the front left microphone is published to a topic named `/soundDetection/signal`. The final specification of this message type will be defined in Task 4.2.3 Sound Detection and Localization.

The names of the topics to be used for each sensor will be read from a data file comprising a sequence of key-value pairs. The key is the name of the sensor. The value is the topic name.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `soundDetection` node is determined by the contents of a configuration file, `soundDetectionConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
<code>algorithm</code>	<code>ITD</code>	Specifies the localization technique to be used.
<code>lowFrequencyCutoff</code>	<code><number></code>	Specifies the low cutoff frequency in the band-pass filter in hertz.
<code>highFrequencyCutoff</code>	<code><number></code>	Specifies the low cutoff frequency in the band-pass filter in hertz.
<code>thresholdEnergy</code>	<code><number></code>	Specifies the threshold energy of the audio signal that qualifies it as a conspicuous sound.
<code>robotTopics</code>	<code>pepperTopics.dat</code>	Specifies the filename of the file in which the physical Pepper robot sensor and actuator topic names are stored.
<code>verboseMode</code>	<code>true, false</code>	Specifies whether diagnostic data is to be printed to the terminal.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node subscribes to two microphone sensor topics. These are specified in the file identified by the `robotTopics` key-value pairs in the configuration file.

The following are the topics to which the `soundDetection` node subscribes.

Topic	Sensor	Platform
<code>/naoqi_driver/audio</code>	<code>MicroFL_sensor</code> , <code>MicroFR_sensor</code> <code>MicroBL_sensor</code> , <code>MicroBR_sensor</code>	Physical robot

Topics Published

The following are the topics to which the `soundDetection` node publishes.

Topic	Sensor / Actuator / Node	Platform
<code>/soundDetection/direction</code>	<code>overtAttention</code> , <code>speechEvent</code>	Physical robot
<code>/soundDetection/signal</code>	<code>speechEvent</code>	Physical robot

Services Supported

This node does not support any services.

Services Called

This node does not call any services.

3.10 Speech Event

ROS Node Name

`speechEvent`

Functional Specification

This ROS node processes a vocal sound, i.e., an utterance, represented as an audio signal, spoken by an interaction partner and transcribes it into written text.

The node uses a deep neural network that has been trained so that it can perform automated speech recognition. In the case that the spoken utterance cannot be recognized, either because the sound is not a spoken utterance or because it uses vocabulary on when the neural network has not been trained, then the node flags this by producing a text that reads “Error: speech not recognized”.

The node has one input: an audio signal that is captured by the `soundDetection` module, published on a topic named `soundDetection/signal`. When the `soundDetection` module is not available or is not publishing to the `soundDetection/signal` topic, the `speechEvent` ROS node flags this by producing a text that reads “Error: soundDetection is down”. The node has one output: a string representing the message in the spoken audio signal.

The node can run in normal mode or verbose mode. In verbose mode, the generated text transcription is printed to the terminal while also being displayed on a graphical application that is automatically opened on the screen (this graphical application is only opened when the ROS node is running in verbose mode).

Speech transcription is triggered by a call made to the `speechEvent/recognise_speech_action` ROS action. This is done via the `speechEvent/recognise_speech_action/goal` ROS topic that is tied to this ROS action. Results are then published on the

`speechEvent/recognise_speech_action/result` ROS topic that is tied to this ROS action.

Configuration File

The operation of the `speechEvent` node is determined by the contents of a configuration file, `speechEventConfiguration.ini`, that contain a list of key-value pairs as shown below.

Key	Values	Effect
language	kinyarwanda, english	Specifies the language in which the utterance is spoken.
model	conformer-transducer, parakeet, whisper	Specifies the ASR model to be used.
verboseMode	true, false	Specifies whether diagnostic data is to be printed to the terminal.
cuda	true, false	Specifies whether to use GPUs. The term ‘cuda’ is chosen as the key to alert the user that only NVIDIA GPUs are supported.
confidence	<number>	The confidence level on a scale of 0 to 1 above which transcriptions are assumed to be acceptable and correct.
vadThreshold	<number>	The confidence level on a scale of 0 to 1 above which an utterance is considered valid in an audio signal.
sampleRate	<number>	Specifies the sampling rate of the incoming audio sourced from the /soundDetection/signal ROS topic.
interUtteranceLen	<number>	The minimum length (in seconds) between two successive utterances, below which the two are assumed to be one utterance.
minUtteranceLen	<number>	The minimum length (in seconds) of an utterance; shorter utterances are ignored and therefore discarded.
maxUtteranceLength	<number>	The maximum length (in seconds) of an utterance, after which any extra utterances are truncated.
heartbeatMsgPeriod	<number>	Specifies the time period in seconds at which a periodic heartbeat message is sent to the terminal.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

The following are the topics to which the `speechEvent` node subscribes.

Topic	Node	Platform
soundDetection/signal	soundDetection	Physical robot

Topics Published

This node does not publish to any ROS topics.

Services Supported

This node provides and advertises a server for a service `/speechEvent/set_language` to set the language to be used. This service is called by the `behaviorController` node.

The following summarizes the service supported.

Service	Message Value	Effect
/speechEvent/set_language	kinyarwanda, english	Sets the transcription language at runtime. Returns a response of 0 if unsuccessful (e.g., if using an unsupported language such as Arabic), and 1 if the service invocation is successful.

Services Called

This node does not call any services.

Actions Supported

The following are the actions that the `speechEvent` node supports.

Action	Node	Effect
/speechEvent/recognise_speech_action	behaviorController	Triggers the speech recognition process to run and transcribe speech utterances

3.11 Tablet Event

ROS Node Name

tabletEvent

Functional Specification

This ROS node provides a ROS service to display a message, which will typically be a menu of interaction options, including a prompt requiring the selection of an option, wait for the visitor to select an option, and return the option selected. This service is called by the `behaviorController` node, in the Robot Mission Interpreter subsystem.

The node has one input: a number indexing the message to be displayed on the tablet PC, passed as an argument to a service call by the `behaviorController`.

The node has one output: a number representing option selected by the visitor, returned by the service call.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `tabletEvent` node is determined by the contents of a configuration file, `tabletEventConfiguration.ini`, that contains a list of key-value pairs as shown below.

Key	Values	Effect
<code>menuDataFile</code>	<code>tabletEventInput.dat</code>	Specifies the filename of the menu data.
<code>verboseMode</code>	<code>true, false</code>	Specifies whether diagnostic data is to be printed to the terminal.

Input Data File

This node reads the data file specified by the `menuDataFile` key-value pair.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node does not subscribe to a ROS topic.

Topics Published

This node does not publish to a ROS topic.

Services Supported

This node provides and advertizes a server for a service `/tabletEvent/prompt_and_get_response` to seek input from the visitor. It uses a package-specific msg, `Prompt.msg` with just one field `string` message. The value of the field is text to be printed on the tablet screen (the format specification is yet to be decided). The node then blocks for a predetermined time, waiting for the visitor to select an option. If the visitor responds within the specified time, the service response is option number $1 - n$; if they don't, the service response is zero. The service is called by the `behaviorController` node, setting the required mode of attention.

The following summarizes the services supported.

Service	Message Value	Effect
/tabletEvent/prompt_and_get_response	<string>	Print message, wait for input, and set response accordingly.

Services Called

This node does not call any services.

3.12 Text to Speech

ROS Node Name

`textToSpeech`

Functional Specification

This ROS node converts English, isiZulu, and Kinyarwanda text to speech. It uses a speech synthesis engine to convert text to an audio file that can then be played on the robots loudspeakers.

The node has one input: a string with the text to be spoken.

The node has one output: an audio signal representing the message in the spoken audio signal.

The node can run in normal mode or verbose mode. In verbose mode, data that is published to topics is also printed to the terminal.

Configuration File

The operation of the `textToSpeech` node is determined by the contents of a configuration file that contain a list of key-value pairs as shown below.

The configuration file is named `textToSpeechConfiguration.ini`.

Key	Values	Effect
language	kinyarwanda, isiZulu, english	Specifies the language in which the text is written.
verboseMode	true, false	Specifies whether diagnostic data is to be printed to the terminal.
ip	172.29.111.230, 172.29.111.240	Specifies the robot you want to use, the ip can be changed according to the network robot is connected to.
port	9559	Specifies a virtual communication endpoint used to identify specific services or processes running on a device / robot.
use_cuda	true, false	Specifies whether to use GPUs. The term 'cuda' is chosen as the key to alert the user that only NVIDIA GPUs are supported.

Input Data File

This node does not read from an input data file.

Output Data File

This node does not write to an output data file.

Topics Subscribed

This node does not subscribe to any topics.

Topics Published

The following are the topics to which the `textToSpeech` node publishes. Note that the `/speech` topic accepts a text string, not an audio signal, and works best with the English language. Alternatives are being investigated.

Topic	Actuator	Platform
/speech	LoudSpeakerLeft, LoudSpeakerRight	Physical robot & Simulator

Services Supported

This node provides and advertizes a server for a service `/textToSpeech/say_text` to request the conversion of a text string to an audio signal, and to play that audio on the robot's loudspeakers.

It uses a standard message type, `std_msgs::String` defined in `std_msgs/String.msg`, to specify the text. If the request is successful, the service response is "1"; if it is unsuccessful, it is "0". The service is called by the `behaviorController` node.

The following summarizes the service supported.

Service	Message Value	Effect
<code>/textToSpeech/say_text</code>	<code>string <message></code>	Convert text to an audio signal and play it on the robot's loudspeakers.
	<code>string, <language></code>	

Services Called

This node does not call any services.

Actions Supported

This node provides and advertises an action server at `/textToSpeech/say_text`, which is used to initiate the conversion of a text string to an audio signal, and to play that audio on the robot's loudspeakers.

It uses a standard message type, `std_msgs::String` defined in `std_msgs/String.msg`, to specify the text. The goal is sent by the `behaviorController` node.

The following summarizes the action servers supported.

Action server	Message Value	Effect
<code>/textToSpeech/say_text</code>	<code>string <message></code>	Convert text to an audio signal and play it on the robot's loudspeakers.
	<code>string, <language></code>	

4 System Architecture in Detail

Figure 2 shows the CSSR4Africa system architecture specified using the ROS nodes, topics, and services specified in Section 3.

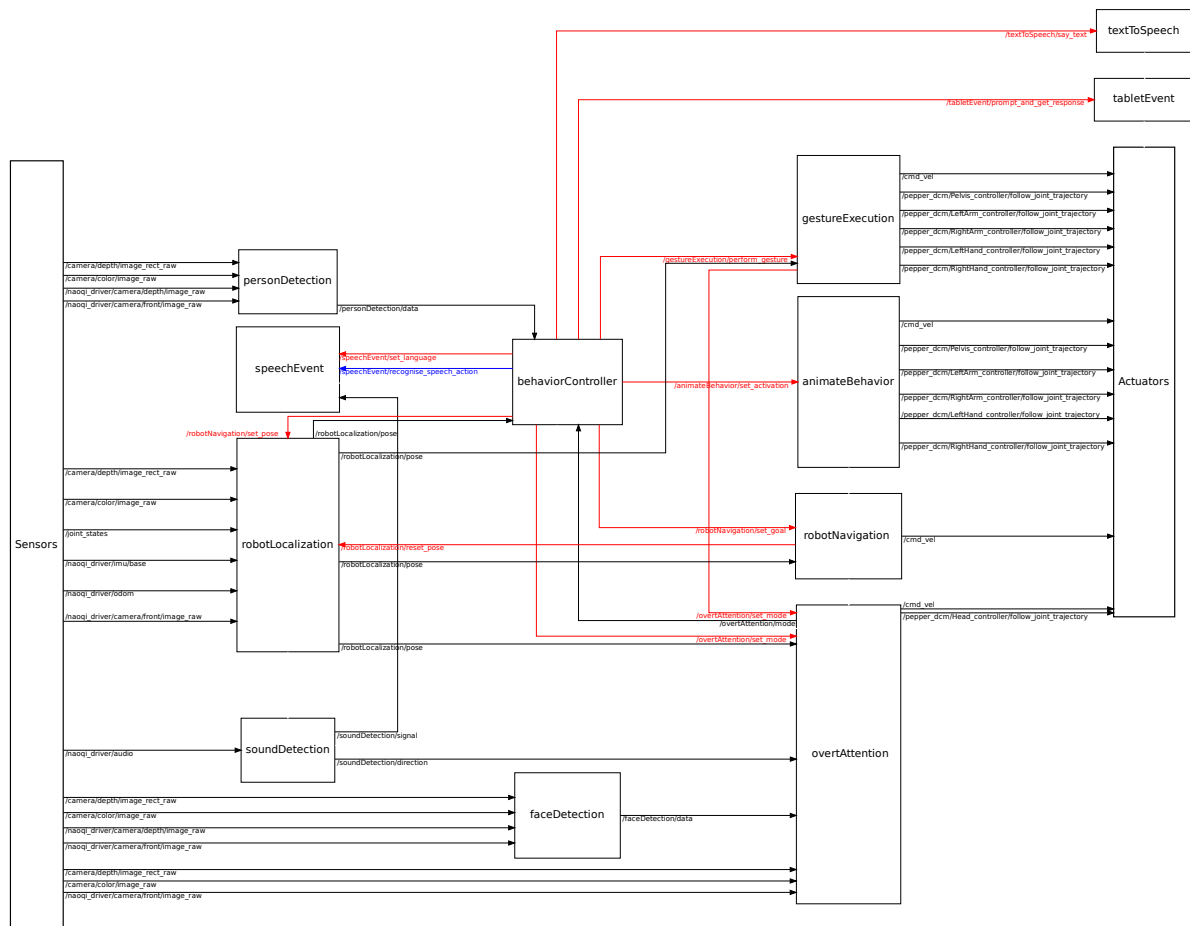


Figure 2: The CSSR4Africa system architecture specified at the level of ROS nodes (thin border rectangles), ROS topics (black lines), ROS services (red lines), ROS actions (blue lines), and sources & sinks for sensor topics and actuator topics (thick border rectangles).

References

- [1] N. D. B. Bruce and J. K. Tsotsos. Saliency, attention, and visual search: An information theoretic approach. *Journal of Vision*, 9(3):1–24, 2009.
- [2] J. K. Tsotsos, S. Culhane, W. Wai, Y. Lai, N. David, and F. Nuflo. Modeling visual attention via selective tuning. *Artificial Intelligence*, 78:507–547, 1995.
- [3] J. K. Tsotsos. Cognitive vision need attention to link sensing with recognition. In H. I. Christensen and H.-H. Nagel, editors, *Cognitive Vision Systems: Sampling the Spectrum of Approaches*, volume 3948 of *LNCS*, pages 25–36, Heidelberg, 2006. Springer.
- [4] J. K. Tsotsos. *A Computational Perspective on Visual Attention*. MIT Press, Cambridge MA, 2011.
- [5] A. Zaharescu, A. L. Rothenstein, and J. K. Tsotsos. Towards a biologically plausible active visual search model. In L. Paletta, J. K. Tsotsos, E. Rome, and G. Humphreys, editors, *Proceedings of the Second International Workshop on Attention and Performance in Computational Vision, WAPCV*, volume *LNCS 3368*, pages 133–147, Berlin, 2004. Springer.

Principal Contributors

The main authors of this deliverable are as follows (in alphabetical order).

Clifford Onyonka, Carnegie Mellon University Africa.

David Vernon, Carnegie Mellon University Africa.

Muhammed Danso, Carnegie Mellon University Africa.

Yohannes Haile, Carnegie Mellon University Africa.

Document History

Version 1.0

First draft.
David Vernon.
24 January 2024.

Version 1.1

Added a configuration file for `tabletEvent`.
David Vernon.
25 January 2024.

Version 1.2

Added key-value pair for `personDetection` and `factDetection` to specify the algorithm to be used.
David Vernon.
01 February 2024.

Version 1.3

Updated topics for the `animateBehavior` and `gestureExecution` nodes, replacing `command`, with `follow_joint_trajectory`. Updated topics for the `soundDetection` node.
Removed `/pepper/camera/stereo/image_raw` topic for simulator in the `faceDetection`, `overtAttention`, `personDetection`, and `robotLocalization` nodes.
Added `behaviorController` to the list of nodes to which `robotLocalization` publishes.
David Vernon.
09 May 2024.

Version 1.4

Updated topics for the `overtAttention` to include a topic on which the mode is published.
Muhammed Danso.
27 August 2024.

Version 1.5

Changed `scriptInterpreter` to `behaviorController`.
Amended some of the Version 1.4 updates.
David Vernon.
28 August 2024.

Version 1.6

Removed `waypointNumber` and `waypointSelection` keys from `robotNavigation`.
Changed the specification of the pose in the `robotNavigation/set_goal` service from using `geometry_msgs/Pose.msg` with `<Point>` `<Quaternion>` to a custom message with `<x>` `<y>` `<theta>`.
Changed references to script language and script interpreter to robot mission language and robot mission interpreter.
David Vernon.
09 September 2024.

Version 1.7

Updated scanning mode for the `overtAttention` to prioritize detected faces when scanning the environment for general features.
Updated `gestureExecution` to include head movement to complement pointing gestures. (with the help of Adedayo)
Updated `faceDetection` to remove eye detection and add mutual gaze detection. (with the help of Yohannes)
Updated Figure 2 system architecture. Muhammed Danso.
21 December 2024.

Version 1.8

Removed `/naoqi_driver/camera/stereo/image_raw` from `overtAttention`, `personDetection`, and `robotLocalization`. Added `/camera/color/image_raw` and `/camera/depth/image_rect_raw` to `overtAttention`, `personDetection`, and `robotLocalization`.

Changed option for selecting camera from `StereoCamera` to `RealSense` in `overtAttention`, `personDetection` and `robotLocalization`.

Updated the `GraphViz` architecture diagram to reflect these changes.

David Vernon.

1 January 2025.

Version 1.9

Changed the configuration file from `faceDetectionConfiguration.ini` to `faceDetectionConfiguration.json`

Updated the face Detection Configuration file key-value pairs table.

Fixed error on the Topics Subscribed overAttention node.

Removed a statement in the Sound Detection regarding message type.

Fixed error on Topics Published on Person Detection.

Yohannes Haile.

16 January 2025.

Version 1.10

Moved `behaviorController` to keep the ROS nodes in alphabetic order, and renamed the Section Robot Mission Interpreter to Behavior Controller.

David Vernon.

20 January 2025.

Version 1.11

Changed Interaction Scenario Manager subsystem to Robot Mission Interpreter subsystem.

Changed Interaction Manager in Figure 1 to Robot Mission Interpreter and added the Environment Knowledge Base.

Removed `knowledgeBase` node.

Added subsections on the `CultureKnowledgeBase` and `EnvironmentKnowledgeBase` helper classes in Sections 3.2 and 3.8, respectively.

David Vernon.

29 January 2025.

Version 2.0

Revised the system architecture to limit the support for the simulator. Simulator support will not be provided for ROS nodes that have the option of using an external device such as the RealSense camera or a LiDAR, or that use the Pepper microphones, speakers, or the tablet PC, or are dependent on data produced by these nodes. Consequently, the only node that supports the simulator is Animate Behavior. Since this is a major revision, the version number is advanced to 2.0.

David Vernon.

1 February 2025.

Version 2.1

Fixed incorrect reference to `personDetection` in documenting the topics to which the `robotNavigation` node publishes.

David Vernon.

9 February 2025.

Version 2.2

Updated the functional descriptions of `soundDetection` and `speechEvent`. Updated the configuration file specification of `speechEvent` and the services it supports.

David Vernon.

26 March 2025.

Version 2.3

Added an option to use compressed images for faceDetection and personDetection.

Added compressed image option in the configuration file table faceDetection and personDetection.

Added topics of the compressed images for the topics Subscribed for faceDetection.

Added topics of the compressed images for the topics Subscribed for personDetection.

Updated the person and face detection configuration file to camel case.

Added width and height as one component of the array of records.

Blue text is used to mark addition of text or table entries and red text was used to mark deletion of text or table entries will be changed to black in the next version.

Yohannes Haile.

31 March 2025.

Version 2.4

Added a service /robotLocalization/set_pose and a call from behaviorController.

Changed behaviour to behavior globally. This mainly affects animateBehavior

Removed the red text in the previous version and changed the blue text to black. David Vernon.

7 May 2025.

Version 2.5

In the configuration file table textToSpeechConfiguration.ini. Added three configurable components which are robot's Ip, port, and use_cuda.

Also added another service request string <language> to the service support table in 3.12.

Muhirwa Richard.

02 June 2025.

Version 2.6

Update speechEvent to publish "Error: soundDetection is down" when soundDetection is unavailable

Open a graphical application that displays text transcriptions on the screen when speechEvent is run in verbose mode

Add confidence, speechPausePeriod and maxUtteranceLength configuration options to speechEvent's configuration file

Add support for the /speechEvent/set_enabled ROS service to speechEvent

Use an updated system architecture diagram to show the new /speechEvent/set_enabled ROS service

Clifford Onyonka.

11 June 2025.

Version 2.7

Updated the Service supported for gestureExecution

Updated the services called for behaviorController

Added Action supported for gestureExecution, robotNavigation, and textToSpeech ROS nodes

Added Action goals sent for behaviorController ROS node

Tsegazeab Tefferi.

26 June 2026.